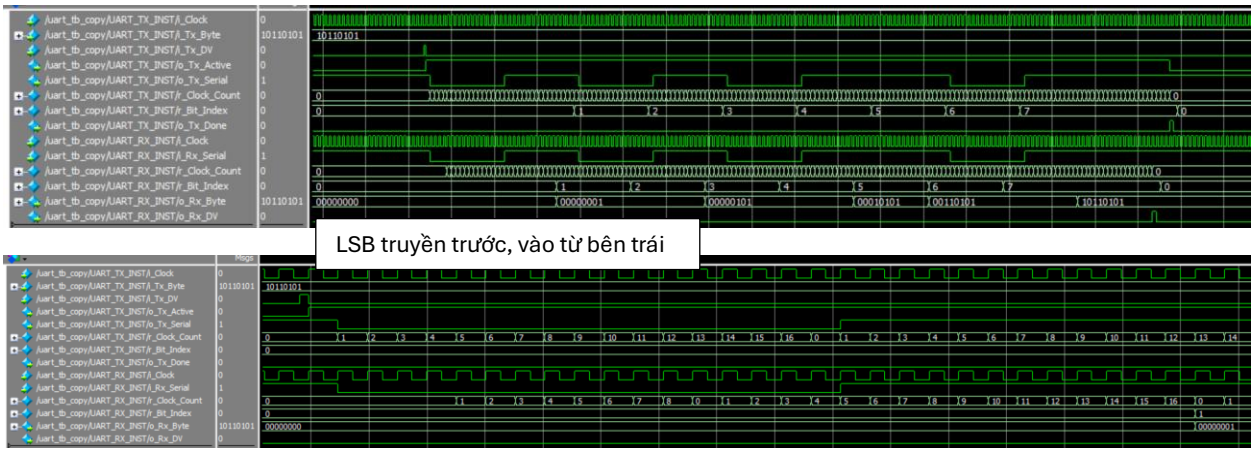
TRƯỚC TIÊN PHẢI BIẾT ĐÂY LÀ UART TRUYỀN THEO KIỂU BẤT ĐỒNG BỘ, NÊN CLOCK Ở ĐÂY LÀ CLOCK TỰ SINH CỦA 2 BÊN GỬI VÀ NHẬN.

CODE HIỆN TẠI LÀ SIMPLEX (CHỈ GIAO TIẾP TỪ 1 CHIỀU)

→ Tx_Serial và Rx_Serial sẽ chung 1 đường do đây là truyền simplex.



$$baud\ rate = \frac{1}{clock\ period(ns) \times 10^{-9}} \times \frac{1}{CLKS_PER_BIT}$$

$$= clock\ frequency \times \frac{1}{CLKS_PER_BIT}$$

$$\Rightarrow 115200 = \frac{1}{510.620915(ns) \times 10^{-9}} \times \frac{1}{17}$$

NÊN XẤP XỈ 510 NS TRONG CODE

c_CLOCK_PERIOD_NS = 510; // 1 clock cycle = 510 ns (số xấp xỉ)

c_CLKS_PER_BIT = 17; // 1 bit cần 17 chu kì clock

c_BIT_PERIOD = 8680; // 1/115200 ≈ 8680ns

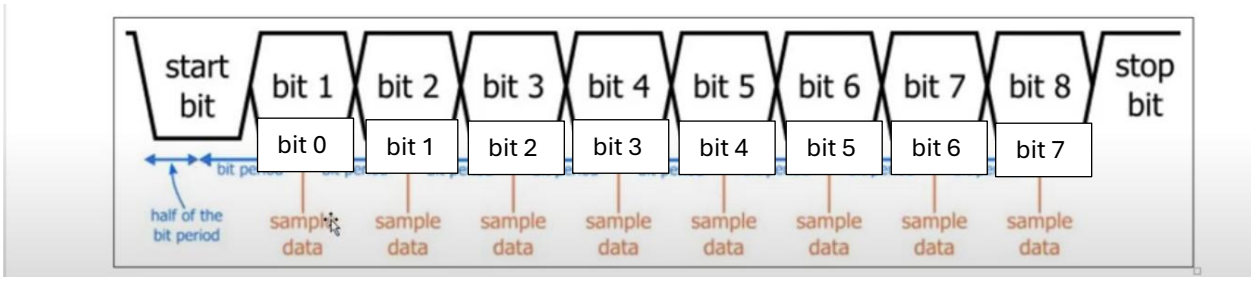
Testbench:

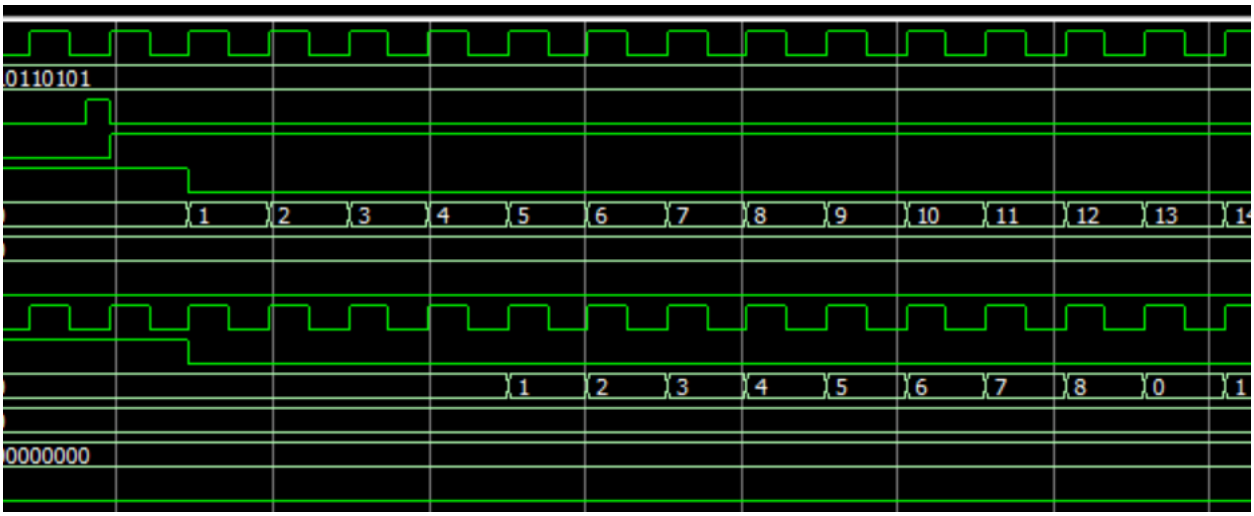
```
r_Tx_DV <= 1'b0;
#(10 * c_BIT_PERIOD); // Tức delay 5 * 8680 = 43400 ns
wait(w_Tx_Done);
#(10 * c_BIT_PERIOD); // Tức delay 5 * 8680 = 43400 ns
```

Hiện tại trong code: 1 start_bit + 8 data_bit + 1 parity_bit + 1 stop_bit = 11 bit

➔ chu kì 11 bit là 11 * 8680 = 95480 ns

95480 + 43400*2 = 182280 ns → mô phỏng 200000 ns là dư dả **(gõ lệnh: run 200000ns)**





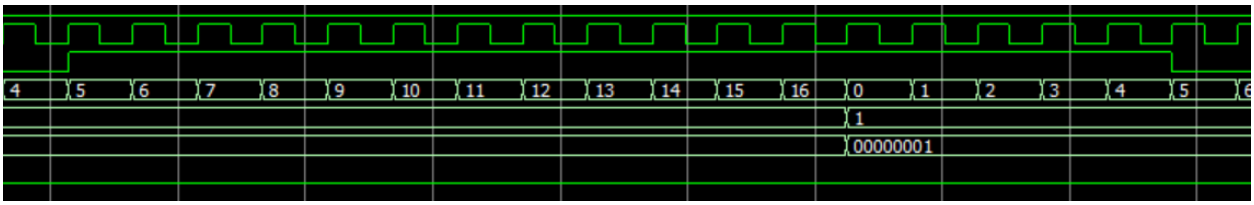
Khi Tx_DV = 1 (trong đúng 1 chu kì clock) tức báo có dữ liệu muốn truyền, lúc này bên Tx sẽ gửi dữ liệu

Nó sẽ gửi Start_bit trước tiên (từ 1 xuống 0) xong mới gửi bit 0 đến bit 7 của data.

khi phát hiện ra Start_bit, nó đợi cho Start_bit ổn định (do FSM) rồi delay nửa chu kì bit (8 chu kì clock), xong mới đếm đủ 1 chu kì bit mới lấy mẫu bit đó.

1 chu kì bit = 17 chu kì clock (1 → 16 → 0)

Lí do là vì ban đầu khởi tạo count_clock = 0, khi start_bit từ 1 xuống 0 thì count_clock mới đếm lên → bắt đầu từ 1, kết thúc tại 16 nên muốn đủ 17 phải tính luôn lúc reset về 0



Mặc dù về lý thuyết, do delay nửa chu kì bit nên lúc bắt đầu lấy mẫu sẽ ngay giữa bit data tức cuối count_clock thứ 8 [(clks_per_bit – 1) / 2] nhưng thực tế, do cơ chế chống nhiễu và FSM:

❓ Vậy tại sao lại có **4 chu kỳ clock trễ** trước khi `r_Clock_Count` bắt đầu tăng?

Câu trả lời nằm ở **đoạn đồng bộ tín hiệu** `i_Rx_Serial` **bằng 2 FF**:

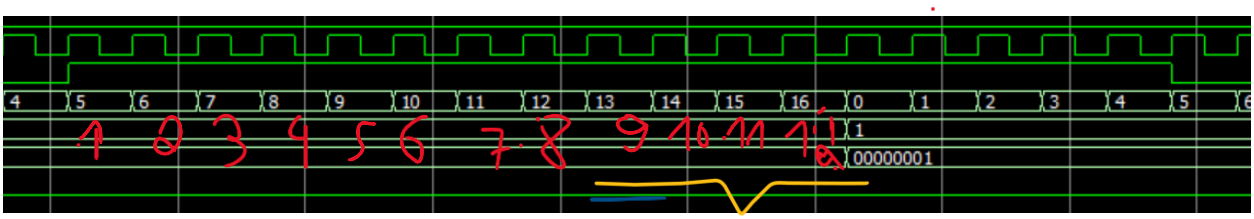
verilog Copy Edit

```verilog
r_Rx_Data_R <= i_Rx_Serial;
r_Rx_Data   <= r_Rx_Data_R;
```

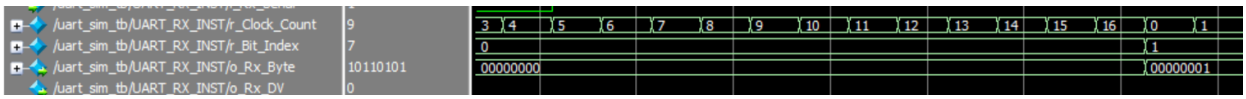
> Khi `i_Rx_Serial` từ 1 → 0 (tức có cạnh xuống), bạn **phải mất 2 chu kỳ clock** để `r_Rx_Data` phản ánh sự thay đổi đó.
> Rồi sau đó mới được FSM thấy trong `s_IDLE`, và **mất thêm 1 chu kỳ để chuyển trạng thái** sang `s_RX_START_BIT`.
> Vào `s_RX_START_BIT`, thì `r_Clock_Count` khởi tạo bằng 0, rồi từ chu kỳ sau mới tăng.

Vậy tổng cộng:
2 chu kỳ delay do đồng bộ + 1 chu kỳ chờ chuyển FSM + 1 chu kỳ đầu tiên `r_Clock_Count` **mới bắt đầu = 4 chu kỳ** như bạn thấy trên waveform.

→ Thời điểm lấy mẫu sẽ là cuối count_clock thứ 8 cộng 4 = cuối count_clock thứ 12



```
s_RX_DATA_BITS:
begin
        if (r_Clock_Count < CLKS_PER_BIT-1)
                r_Clock_Count <= r_Clock_Count + 1;
        else
        begin
                r_Clock_Count <= 0;
                r_Rx_Byte[r_Bit_Index] <= r_Rx_Data;
                if (r_Bit_Index < 7)
                        r_Bit_Index <= r_Bit_Index + 1;
                else
                        r_SM_Main <= s_RX_STOP_BIT;
        end
end
```

Theo đoạn code trên, nếu clock_count == clk_per_bit – 1 (17-1=16) thì reset clock_count về 0, và gán Rx_Data vào Rx_Byte(Bit_index).

Lẽ ra khi BẮT ĐẦU clock 16, phải gán bit data vô Rx_byte, nhưng trong hình CUỐI clock 16 và đầu clock 0 (sau khi reset) mới gán bit data. Lí do 0là xài phép gán **non-blocking** nên PHẢI ĐỢI HẾT CHU KÌ XUNG CLOCK HIỆN TẠI, VÀ BẮT ĐẦU CẠNH LỆNH XUNG CLOCK KẾ TIẾP mới cập nhật giá trị mới chứ không cập nhật ngay.